

SQL 練習問題集

ECファッションサイト「StyleHub」のデータを使った、SQL中級～上級の92問。CASE WHEN・ウィンドウ関数・CTE・名寄せまで、実務に必要な構文をひと通り。

92 **15** **4** **1,321**
全問題数 トピック テーブル サンプル行数

DATA ENVIRONMENT 4 テーブル / 1,321 行

データ環境

ECファッションサイト「StyleHub」の架空データ。**Phase 1～2** では `items` / `users` / `sales_records` の3テーブル、**Phase 3**（名寄せ）では追加で `users_raw` を使います。

items

30件

商品マスタ

カラム	型	説明
id	<code>int</code>	商品ID
name	<code>string</code>	商品名
gender	<code>int</code>	0:メンズ・1:レディース・2:ユニセックス
price	<code>int</code>	販売価格
cost	<code>int</code>	原価

users

325件

ユーザーマスタ

カラム	型	説明
id	<code>int</code>	ユーザーID
name	<code>string</code>	氏名
gender	<code>int</code>	0 または 1
age	<code>int</code>	年齢



sales_records

910件

販売記録

カラム	型	説明
id	int	販売記録ID
user_id	int	ユーザーID (→ <code>users.id</code>)
item_id	int	商品ID (→ <code>items.id</code>)
purchased_at	date	購入日



users_raw

56件

名寄せ用 **Phase 3** 重複・表記ゆれ・NULLを含む生データ

カラム	型	説明
record_id	int	レコードID (一意)
name	string	氏名 (表記ゆれあり)
email	string	メールアドレス (NULLあり)
phone	string	電話番号 (NULL・ハイフン有無あり)
source	string	データソース ('CRM' / 'Form' / 'CSV')
updated_at	timestamp	更新日時



表記ルール

- 日本語のエイリアスは `"～"` (ダブルクォート) で囲む
- `purchased_at` は DATE 型で日付演算が可能
- SQL 例は **PostgreSQL / Snowflake** 系の構文がベース

PHASE 1 0 / 26 問



Phase 1 : 最優先トピック



トピック A : CASE WHEN (条件分岐)

A-1

年齢を年代カテゴリに変換

基礎

users テーブルから、ユーザー名・年齢・年代カテゴリを取得してください。年代は以下の通り：

- 19歳以下 → 「10代以下」
- 20～29歳 → 「20代」

- 30～39歳 → 「30代」
- 40～49歳 → 「40代」
- 50歳以上 → 「50代以上」

▼ 解答

```
SELECT
  name AS "ユーザー名",
  age AS "年齢",
  CASE
    WHEN age < 20 THEN '10代以下'
    WHEN age < 30 THEN '20代'
    WHEN age < 40 THEN '30代'
    WHEN age < 50 THEN '40代'
    ELSE '50代以上'
  END AS "年代"
FROM users;
```

SQL

ポイント：CASE WHEN の条件は上から順に評価される。`age < 20` が真なら以降の WHEN は評価されないで、条件は **小さい順 or 大きい順** に並べると安全。

A-2 価格帯ラベルを付ける

基礎

items テーブルから、商品名・販売価格・価格帯ラベルを取得してください。価格帯は：

- 1,000円未満 → 「低価格」
- 1,000円以上 3,000円未満 → 「中価格」
- 3,000円以上 → 「高価格」

▼ 解答

```
SELECT
  name AS "商品名",
  price AS "販売価格",
  CASE
    WHEN price < 1000 THEN '低価格'
    WHEN price < 3000 THEN '中価格'
    ELSE '高価格'
  END AS "価格帯"
FROM items;
```

SQL

A-3 性別コードを文字列に変換

基礎

items テーブルから、商品名・性別ラベルを取得してください。性別は：

- 0 → 「メンズ」
- 1 → 「レディース」
- 2 → 「ユニセックス」

▼ 解答

```
SELECT
  name AS "商品名",
  CASE gender
    WHEN 0 THEN 'メンズ'
    WHEN 1 THEN 'レディース'
    WHEN 2 THEN 'ユニセックス'
  END AS "性別ラベル"
FROM items;
```

SQL

ポイント： `CASE 列名 WHEN 値 THEN ...` の **シンプルCASE構文** が使える。

A-4 在庫状況の表示切替

基礎

仮に items に `stock`（在庫数）カラムがあるとして、在庫数で以下のラベルを付けてください：

- 0 → 「在庫切れ」
- 1～5 → 「残りわずか」
- 6以上 → 「在庫あり」

（実データには `stock` がないので、`price` を在庫数とみなして書いてください）

▼ 解答

```
SELECT
  name AS "商品名",
  price AS "仮在庫数",
  CASE
    WHEN price = 0 THEN '在庫切れ'
    WHEN price <= 5 THEN '残りわずか'
    ELSE '在庫あり'
  END AS "在庫ステータス"
FROM items;
```

SQL

A-5 ORDER BY での独自ソート順

基礎

items テーブルから商品名と性別ラベルを取得し、「メンズ → レディース → ユニセックス」の順で並べてください。

▼ 解答

```
SELECT
  name AS "商品名",
  CASE gender
    WHEN 0 THEN 'メンズ'
    WHEN 1 THEN 'レディース'
    WHEN 2 THEN 'ユニセックス'
  END AS "性別ラベル"
FROM items
ORDER BY
  CASE gender
    WHEN 0 THEN 1
    WHEN 1 THEN 2
    WHEN 2 THEN 3
  END;
```

ポイント : ORDER BY に CASE を書くことで **任意のソート順** を作れる。

A-6 性別ごとの商品数をクロス集計

応用

items テーブルから、性別ごとの商品数を **1行に並べて** 表示してください（クロス集計）。

▼ 解答

```
SELECT
  SUM(CASE WHEN gender = 0 THEN 1 ELSE 0 END) AS "メンズ商品数",
  SUM(CASE WHEN gender = 1 THEN 1 ELSE 0 END) AS "レディース商品数",
  SUM(CASE WHEN gender = 2 THEN 1 ELSE 0 END) AS "ユニセックス商品数"
FROM items;
```

ポイント : これが **条件付き集計**（クロス集計の**基礎**）。「条件に合えば1、合わなければ0」を SUM することで件数を出せる。

A-7 年代別×性別の購入金額クロス集計

応用

users × sales_records × items を結合し、年代別・性別ごとの購入金額を集計してください。

▼ 解答

```
SELECT
CASE
  WHEN users.age < 20 THEN '10代以下'
  WHEN users.age < 30 THEN '20代'
  WHEN users.age < 40 THEN '30代'
  WHEN users.age < 50 THEN '40代'
  ELSE '50代以上'
END AS "年代",
SUM(CASE WHEN users.gender = 0 THEN items.price ELSE 0 END) AS "メンズ購入額",
SUM(CASE WHEN users.gender = 1 THEN items.price ELSE 0 END) AS "レディース購入額"
FROM users
INNER JOIN sales_records ON users.id = sales_records.user_id
INNER JOIN items ON sales_records.item_id = items.id
GROUP BY
CASE
  WHEN users.age < 20 THEN '10代以下'
  WHEN users.age < 30 THEN '20代'
  WHEN users.age < 40 THEN '30代'
  WHEN users.age < 50 THEN '40代'
  ELSE '50代以上'
END
ORDER BY "年代";
```

SQL

A-8 商品ごとの粗利率を帯ラベル化

応用

items テーブルから、商品名・粗利率（粗利÷販売価格）の帯ラベルを取得してください：

- 30%以上 → 「高利益」
- 15%以上 30%未満 → 「中利益」
- 15%未満 → 「低利益」

▼ 解答

```
SELECT
name AS "商品名",
(price - cost) * 100 / price AS "粗利率(%)",
CASE
  WHEN (price - cost) * 100 / price >= 30 THEN '高利益'
  WHEN (price - cost) * 100 / price >= 15 THEN '中利益'
  ELSE '低利益'
END AS "利益区分"
FROM items;
```

SQL

A-9 顧客ステータス分類

応用

各ユーザーの購入回数によって、以下のステータスに分類してください：

- 0回 → 「未購入」
- 1〜2回 → 「ライト」
- 3〜5回 → 「ミドル」
- 6回以上 → 「ヘビー」

▼ 解答

```
SELECT
  users.name AS "ユーザー名",
  COUNT(sales_records.id) AS "購入回数",
  CASE
    WHEN COUNT(sales_records.id) = 0 THEN '未購入'
    WHEN COUNT(sales_records.id) <= 2 THEN 'ライト'
    WHEN COUNT(sales_records.id) <= 5 THEN 'ミドル'
    ELSE 'ヘビー'
  END AS "ステータス"
FROM users
LEFT JOIN sales_records ON users.id = sales_records.user_id
GROUP BY users.id, users.name
ORDER BY COUNT(sales_records.id) DESC;
```

SQL

トピック B：日付関数

B-1 今日の日付を取得

基礎

「今日」の日付を取得してください。

▼ 解答

```
SELECT CURRENT_DATE AS "今日";
```

SQL

または：

```
SELECT CURRENT_TIMESTAMP AS "現在日時";
```

SQL

B-2 30日前の日付を取得

基礎

今日から30日前の日付を取得してください。

▼ 解答

```
SELECT CURRENT_DATE - INTERVAL '30 days' AS "30日前";
```

SQL

または DBによって：

```
SELECT DATEADD(day, -30, CURRENT_DATE) AS "30日前";
```

SQL

B-3 purchased_at から年・月・日を取り出す

基礎

sales_records から、購入記録ID・購入日・購入年・購入月・購入日(日付の日)を取得してください。

▼ 解答

```
SELECT
  id AS "販売記録ID",
  purchased_at AS "購入日",
  EXTRACT(YEAR FROM purchased_at) AS "購入年",
  EXTRACT(MONTH FROM purchased_at) AS "購入月",
  EXTRACT(DAY FROM purchased_at) AS "購入日(日)"
FROM sales_records;
```

SQL

B-4 月初に丸める (DATE_TRUNC)

基礎

sales_records から、購入日と「その月の初日」を取得してください。

▼ 解答

```
SELECT
  purchased_at AS "購入日",
  DATE_TRUNC('month', purchased_at) AS "月初"
FROM sales_records;
```

SQL

ポイント： `DATE_TRUNC('month', ...)` で「2024-07-15」→「2024-07-01」のように丸められる。

B-5 2つの日付の差を計算

基礎

各販売記録について、「2018-01-01からの経過日数」を計算してください。

▼ 解答

```
SELECT
  id AS "販売記録ID",
  purchased_at AS "購入日",
  purchased_at - DATE '2018-01-01' AS "経過日数"
FROM sales_records;
```

SQL

B-6 月次売上トレンド

応用

`sales_records` × `items` を結合し、月ごとの売上総額を集計してください。

▼ 解答

```
SELECT
  DATE_TRUNC('month', sales_records.purchased_at) AS "月",
  SUM(items.price) AS "月次売上額"
FROM sales_records
INNER JOIN items ON sales_records.item_id = items.id
GROUP BY DATE_TRUNC('month', sales_records.purchased_at)
ORDER BY "月" ASC;
```

SQL

B-7 直近30日間の購入者数

応用

`sales_records` の最新の購入日を基準に、その日から **30日以内** に購入したユニークユーザー数を取得してください。

▼ 解答

```
SELECT
  COUNT(DISTINCT user_id) AS "直近30日購入者数"
FROM sales_records
WHERE purchased_at >= (
  SELECT MAX(purchased_at) - INTERVAL '30 days'
```

SQL

```
FROM sales_records  
);
```

B-8 各ユーザーの最終購入日からの経過日数

応用

各ユーザーの最終購入日と、データ最新日からの経過日数を取得してください。

▼ 解答

```
SELECT  
  users.name AS "ユーザー名",  
  MAX(sales_records.purchased_at) AS "最終購入日",  
  (SELECT MAX(purchased_at) FROM sales_records) - MAX(sales_records.purchased_at) AS  
  "経過日数"  
FROM users  
INNER JOIN sales_records ON users.id = sales_records.user_id  
GROUP BY users.id, users.name  
ORDER BY "経過日数" DESC;
```

SQL

B-9 曜日別の販売件数

応用

sales_records から、曜日ごとの販売件数を取得してください。

▼ 解答

```
SELECT  
  EXTRACT(DOW FROM purchased_at) AS "曜日番号",  
  CASE EXTRACT(DOW FROM purchased_at)  
    WHEN 0 THEN '日曜'  
    WHEN 1 THEN '月曜'  
    WHEN 2 THEN '火曜'  
    WHEN 3 THEN '水曜'  
    WHEN 4 THEN '木曜'  
    WHEN 5 THEN '金曜'  
    WHEN 6 THEN '土曜'  
  END AS "曜日",  
  COUNT(*) AS "販売件数"  
FROM sales_records  
GROUP BY EXTRACT(DOW FROM purchased_at)  
ORDER BY "曜日番号";
```

SQL

📘 トピック C：ウィンドウ関数の基礎

C-1 全商品に売上順位を付ける (RANK)

基礎

商品ごとの売上額を集計し、売上ランキング順位を付けてください。

▼ 解答

```
SELECT
  items.name AS "商品名",
  SUM(items.price) AS "売上額",
  RANK() OVER (ORDER BY SUM(items.price) DESC) AS "順位"
FROM items
INNER JOIN sales_records ON items.id = sales_records.item_id
GROUP BY items.id, items.name
ORDER BY "順位";
```

SQL

C-2 ユーザーごとに購入順を振る (ROW_NUMBER)

基礎

sales_records の各レコードに、ユーザーごとの購入順番号 を付けてください。

▼ 解答

```
SELECT
  user_id AS "ユーザーID",
  purchased_at AS "購入日",
  ROW_NUMBER() OVER (
    PARTITION BY user_id
    ORDER BY purchased_at
  ) AS "購入順"
FROM sales_records;
```

SQL

C-3 性別ごとの売上ランキング (PARTITION BY)

基礎

性別カテゴリごとに、商品の売上ランキングを付けてください。

▼ 解答

SQL

```
SELECT
  items.gender AS "性別",
  items.name AS "商品名",
  SUM(items.price) AS "売上額",
  RANK() OVER (
    PARTITION BY items.gender
    ORDER BY SUM(items.price) DESC
  ) AS "性別内順位"
FROM items
INNER JOIN sales_records ON items.id = sales_records.item_id
GROUP BY items.id, items.name, items.gender
ORDER BY items.gender, "性別内順位";
```

C-4 RANK と DENSE_RANK の違い

基礎

商品の売上ランキングを `RANK()` と `DENSE_RANK()` の両方で付けて、違いを比較してください。

▼ 解答

SQL

```
SELECT
  items.name AS "商品名",
  SUM(items.price) AS "売上額",
  RANK() OVER (ORDER BY SUM(items.price) DESC) AS "RANK",
  DENSE_RANK() OVER (ORDER BY SUM(items.price) DESC) AS "DENSE_RANK"
FROM items
INNER JOIN sales_records ON items.id = sales_records.item_id
GROUP BY items.id, items.name
ORDER BY "RANK";
```

ポイント : RANK は同点で次の順位が飛び (1, 2, 2, 4, ...) 、DENSE_RANK は飛ばない (1, 2, 2, 3, ...) 。

C-5 性別ごとの売上TOP3商品を抽出

応用

性別ごとに、売上TOP3 の商品だけを抽出してください。

▼ 解答

SQL

```
SELECT *
FROM (
  SELECT
    items.gender AS "性別",
    items.name AS "商品名",
    SUM(items.price) AS "売上額",
```

```

RANK() OVER (
  PARTITION BY items.gender
  ORDER BY SUM(items.price) DESC
) AS "順位"
FROM items
INNER JOIN sales_records ON items.id = sales_records.item_id
GROUP BY items.id, items.name, items.gender
) AS ranked
WHERE "順位" <= 3
ORDER BY "性別", "順位";

```

ポイント：ウィンドウ関数の結果を WHERE で絞り込むには、**サブクエリで一旦囲む**必要がある。

C-6 各ユーザーの最高購入金額の取引情報

応用

各ユーザーが行った購入のうち、最も高額な商品の購入レコードを1件だけ取得してください。

▼ 解答

```

SELECT *
FROM (
  SELECT
    users.name AS "ユーザー名",
    items.name AS "商品名",
    items.price AS "購入金額",
    sales_records.purchased_at AS "購入日",
    ROW_NUMBER() OVER (
      PARTITION BY users.id
      ORDER BY items.price DESC
    ) AS rn
  FROM users
  INNER JOIN sales_records ON users.id = sales_records.user_id
  INNER JOIN items ON sales_records.item_id = items.id
) AS ranked
WHERE rn = 1;

```

SQL

C-7 商品ごとの累計購入回数推移

応用

各商品の販売記録に、その商品の購入が何回目かを付けてください。

▼ 解答

```
SELECT
  items.name AS "商品名",
  sales_records.purchased_at AS "購入日",
  ROW_NUMBER() OVER (
    PARTITION BY items.id
    ORDER BY sales_records.purchased_at
  ) AS "累計購入回数"
FROM items
INNER JOIN sales_records ON items.id = sales_records.item_id
ORDER BY items.name, sales_records.purchased_at;
```

SQL

Phase 1 総合問題

P1-総合 年代×性別×期間のクロス分析

総合

`users` × `sales_records` × `items` を結合し、年代・性別・直近6ヶ月／6～12ヶ月前の購入額・年代内順位を出してください。

▼ 解答

```
WITH base AS (
  SELECT
    CASE
      WHEN users.age < 20 THEN '10代以下'
      WHEN users.age < 30 THEN '20代'
      WHEN users.age < 40 THEN '30代'
      WHEN users.age < 50 THEN '40代'
      ELSE '50代以上'
    END AS "年代",
    CASE users.gender WHEN 0 THEN '男性' WHEN 1 THEN '女性' END AS "性別",
    sales_records.purchased_at,
    items.price
  FROM users
  INNER JOIN sales_records ON users.id = sales_records.user_id
  INNER JOIN items ON sales_records.item_id = items.id
),
max_date AS (SELECT MAX(purchased_at) AS latest FROM sales_records)
SELECT
  base."年代",
  base."性別",
  SUM(CASE WHEN base.purchased_at >= max_date.latest - INTERVAL '6 months' THEN
base.price ELSE 0 END) AS "直近6ヶ月購入額",
  SUM(CASE WHEN base.purchased_at < max_date.latest - INTERVAL '6 months'
AND base.purchased_at >= max_date.latest - INTERVAL '12 months' THEN
base.price ELSE 0 END) AS "6_12ヶ月前購入額",
```

SQL

```
RANK() OVER (PARTITION BY base."年代", base."性別" ORDER BY SUM(CASE WHEN
base.purchased_at >= max_date.latest - INTERVAL '6 months' THEN base.price ELSE 0
END) DESC) AS "年代内順位"
FROM base, max_date
GROUP BY base."年代", base."性別", max_date.latest
ORDER BY base."年代", base."性別";
```

Phase 2 : 高優先トピック

トピック D : NULL処理 (COALESCE/NULLIF)

D-1 NULL を 0 に置換

基礎

仮に `cost` が NULL のレコードがあるとして、NULL の場合は 0 として扱って表示してください。

▼ 解答

```
SELECT
  name AS "商品名",
  COALESCE(cost, 0) AS "原価"
FROM items;
```

SQL

ポイント : `COALESCE(値, デフォルト)` は「値が NULL でなければ値、NULL ならデフォルト」を返す。

D-2 NULL を「未設定」に置換

基礎

仮に `users.name` が NULL のユーザーがいるとして、その場合は「未設定」と表示してください。

▼ 解答

```
SELECT
  id AS "ユーザーID",
  COALESCE(name, '未設定') AS "ユーザー名"
FROM users;
```

SQL

D-3 ゼロ除算防止 (NULLIF)

基礎

`price / cost` で売価倍率を計算したいが、`cost` が 0 のときにエラーにならないようにしてください。

▼ 解答

SQL

```
SELECT
  name AS "商品名",
  price AS "販売価格",
  cost AS "原価",
  price / NULLIF(cost, 0) AS "売価倍率"
FROM items;
```

ポイント: `NULLIF(cost, 0)` は cost が 0 なら NULL を返す。

D-4 複数候補から最初の非NULLを採用

基礎

仮に `users.name` が NULL の場合は `id` を表示してください。

▼ 解答

SQL

```
SELECT
  id AS "ユーザーID",
  COALESCE(name, CAST(id AS VARCHAR)) AS "表示名"
FROM users;
```

ポイント: `COALESCE(a, b, c, d)` のように複数引数で「最初の非NULL値」を返せる。

D-5 LEFT JOIN後の購入回数を NULL → 0 に

応用

`users` を起点に LEFT JOIN し、購入回数を集計してください。購入0件のユーザーは `0` として表示。

▼ 解答

SQL

```
SELECT
  users.name AS "ユーザー名",
  COUNT(sales_records.id) AS "購入回数",
  COALESCE(SUM(items.price), 0) AS "累計購入金額"
FROM users
LEFT JOIN sales_records ON users.id = sales_records.user_id
LEFT JOIN items ON sales_records.item_id = items.id
GROUP BY users.id, users.name
ORDER BY "累計購入金額" DESC;
```

D-6 安全な売価倍率計算

応用

原価が0 または NULL の場合は売価倍率を `NULL` のままにし、それ以外は `price * 100 / cost` で計算してください。

▼ 解答

```
SELECT
  name AS "商品名",
  price AS "販売価格",
  COALESCE(cost, 0) AS "原価",
  price * 100 / NULLIF(cost, 0) AS "売価倍率(%)"
FROM items;
```

SQL

D-7 メールアドレス > 電話 > 会員番号の優先採用

応用

`users_raw` から、ユーザーを識別するキーとして「メール → 電話 → ID」の優先順で取得してください。

▼ 解答

```
SELECT
  record_id,
  name,
  COALESCE(email, phone, CAST(record_id AS VARCHAR)) AS "識別キー"
FROM users_raw;
```

SQL

トピック E：ウィンドウ関数の応用

E-1 前回の購入日を取得 (LAG)

基礎

各ユーザーの各購入レコードについて、前回の購入日を取得してください。

▼ 解答

```
SELECT
  user_id AS "ユーザーID",
  purchased_at AS "購入日",
  LAG(purchased_at) OVER (
```

SQL

```
    PARTITION BY user_id
    ORDER BY purchased_at
) AS "前回購入日"
FROM sales_records;
```

E-2 次回の購入日を取得 (LEAD)

基礎

各購入レコードに、次回の購入日を付けてください。

▼ 解答

```
SELECT
    user_id AS "ユーザーID",
    purchased_at AS "購入日",
    LEAD(purchased_at) OVER (
        PARTITION BY user_id
        ORDER BY purchased_at
    ) AS "次回購入日"
FROM sales_records;
```

SQL

E-3 累計売上の計算

基礎

`sales_records` × `items` の各レコードに、購入日順の **累計売上額** を付けてください。

▼ 解答

```
SELECT
    sales_records.purchased_at AS "購入日",
    items.price AS "購入額",
    SUM(items.price) OVER (
        ORDER BY sales_records.purchased_at
    ) AS "累計売上"
FROM sales_records
INNER JOIN items ON sales_records.item_id = items.id
ORDER BY sales_records.purchased_at;
```

SQL

E-4 7日間移動平均

基礎

日次売上の7日間移動平均を計算してください。

▼ 解答

```
WITH daily AS (  
  SELECT  
    sales_records.purchased_at AS "日付",  
    SUM(items.price) AS "日次売上"  
  FROM sales_records  
  INNER JOIN items ON sales_records.item_id = items.id  
  GROUP BY sales_records.purchased_at  
)  
SELECT  
  "日付",  
  "日次売上",  
  AVG("日次売上") OVER (  
    ORDER BY "日付"  
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW  
  ) AS "7日移動平均"  
FROM daily  
ORDER BY "日付";
```

SQL

E-5 グループ内の累計

基礎

各ユーザーごとの購入累計金額を、購入日順に計算してください。

▼ 解答

```
SELECT  
  sales_records.user_id AS "ユーザーID",  
  sales_records.purchased_at AS "購入日",  
  items.price AS "購入額",  
  SUM(items.price) OVER (  
    PARTITION BY sales_records.user_id  
    ORDER BY sales_records.purchased_at  
  ) AS "ユーザー累計"  
FROM sales_records  
INNER JOIN items ON sales_records.item_id = items.id  
ORDER BY sales_records.user_id, sales_records.purchased_at;
```

SQL

E-6 ユーザーごとの購入間隔分析

応用

各購入レコードに、前回購入日からの経過日数を付けてください。

▼ 解答

```
SELECT
  user_id AS "ユーザーID",
  purchased_at AS "購入日",
  LAG(purchased_at) OVER (
    PARTITION BY user_id
    ORDER BY purchased_at
  ) AS "前回購入日",
  purchased_at - LAG(purchased_at) OVER (
    PARTITION BY user_id
    ORDER BY purchased_at
  ) AS "購入間隔(日)"
FROM sales_records
ORDER BY user_id, purchased_at;
```

SQL

E-7 月次累計売上推移

応用

月別売上を集計し、月次累計売上を計算してください。

▼ 解答

```
WITH monthly AS (
  SELECT
    DATE_TRUNC('month', sales_records.purchased_at) AS "月",
    SUM(items.price) AS "月次売上"
  FROM sales_records
  INNER JOIN items ON sales_records.item_id = items.id
  GROUP BY DATE_TRUNC('month', sales_records.purchased_at)
)
SELECT
  "月",
  "月次売上",
  SUM("月次売上") OVER (ORDER BY "月") AS "累計売上"
FROM monthly
ORDER BY "月";
```

SQL

E-8 前月比成長率

応用

月次売上の前月比成長率（％）を計算してください。

▼ 解答

```
WITH monthly AS (  
    SELECT  
        DATE_TRUNC('month', sales_records.purchased_at) AS "月",  
        SUM(items.price) AS "月次売上"  
    FROM sales_records  
    INNER JOIN items ON sales_records.item_id = items.id  
    GROUP BY DATE_TRUNC('month', sales_records.purchased_at)  
)  
SELECT  
    "月",  
    "月次売上",  
    LAG("月次売上") OVER (ORDER BY "月") AS "前月売上",  
    ("月次売上" - LAG("月次売上") OVER (ORDER BY "月")) * 100 / NULLIF(LAG("月次売上") OVER  
    (ORDER BY "月"), 0) AS "前月比(%)"  
FROM monthly  
ORDER BY "月";
```

SQL

E-9 各取引が「初回」「リピート」かをラベル付け

応用

各購入レコードに、ユーザーの「初回購入」か「リピート購入」かのラベルを付けてください。

▼ 解答

```
SELECT  
    user_id AS "ユーザーID",  
    purchased_at AS "購入日",  
    CASE  
        WHEN ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY purchased_at) = 1  
        THEN '初回'  
        ELSE 'リピート'  
    END AS "区分"  
FROM sales_records  
ORDER BY user_id, purchased_at;
```

SQL

トピック F : CTE (WITH句)

F-1 1つの WITH 句を使った基本クエリ

基礎

商品ごとの販売件数を CTE で計算し、その結果から販売件数が 30 件以上の商品を取得してください。

▼ 解答

```
WITH item_sales AS (SQL
  SELECT
    items.id,
    items.name AS "商品名",
    COUNT(*) AS "販売件数"
  FROM items
  INNER JOIN sales_records ON items.id = sales_records.item_id
  GROUP BY items.id, items.name
)
SELECT *
FROM item_sales
WHERE "販売件数" >= 30
ORDER BY "販売件数" DESC;
```

F-2 中間集計を CTE で名前付け

基礎

ユーザーごとの累計購入額を CTE で計算し、平均購入額を超えるユーザーを取得してください。

▼ 解答

```
WITH user_purchase AS (SQL
  SELECT
    users.id,
    users.name AS "ユーザー名",
    SUM(items.price) AS "累計購入額"
  FROM users
  INNER JOIN sales_records ON users.id = sales_records.user_id
  INNER JOIN items ON sales_records.item_id = items.id
  GROUP BY users.id, users.name
)
SELECT *
FROM user_purchase
WHERE "累計購入額" > (SELECT AVG("累計購入額") FROM user_purchase)
ORDER BY "累計購入額" DESC;
```

F-3 既存のサブクエリを CTE に書き換え

基礎

サブクエリを CTE で書き直してください。

▼ 解答

```
WITH avg_price AS (  
  SELECT AVG(price) AS avg_p  
  FROM items  
)  
SELECT  
  items.name AS "商品名",  
  items.price AS "販売価格"  
FROM items, avg_price  
WHERE items.price > avg_price.avg_p;
```

SQL

F-4 複数の CTE を連鎖

基礎

商品ごとの売上を集計 → 平均を計算 → 平均を超える商品を抽出。

▼ 解答

```
WITH item_sales AS (  
  SELECT items.id, items.name, SUM(items.price) AS sales  
  FROM items  
  INNER JOIN sales_records ON items.id = sales_records.item_id  
  GROUP BY items.id, items.name  
)  
avg_sales AS (  
  SELECT AVG(sales) AS avg_value FROM item_sales  
)  
SELECT item_sales.*  
FROM item_sales, avg_sales  
WHERE item_sales.sales > avg_sales.avg_value  
ORDER BY item_sales.sales DESC;
```

SQL

F-5 過去の Q10 を CTE で書き直し

応用

「平均売上を上回る商品」の問題を CTE で書き直してください。

▼ 解答

```
WITH item_sales AS (  
  SELECT items.id, items.name AS "商品名", SUM(items.price) AS "売上額"  
  FROM items
```

SQL


```

INNER JOIN sales_records ON items.id = sales_records.item_id
GROUP BY items.id, items.name
),
avg_sales AS (SELECT AVG("売上額") AS avg_value FROM item_sales)
SELECT item_sales."商品名", item_sales."売上額"
FROM item_sales, avg_sales
WHERE item_sales."売上額" > avg_sales.avg_value
ORDER BY item_sales."売上額" DESC;

```

F-6 ユーザーセグメント分析

応用

累計購入額に応じて「VIP/優良/一般」を分類し、セグメント別の人数と合計購入額を集計。

▼ 解答

```

WITH user_total AS (
    SELECT users.id, SUM(items.price) AS total_amount
    FROM users
    INNER JOIN sales_records ON users.id = sales_records.user_id
    INNER JOIN items ON sales_records.item_id = items.id
    GROUP BY users.id
),
segmented AS (
    SELECT
        id, total_amount,
        CASE
            WHEN total_amount >= 50000 THEN 'VIP'
            WHEN total_amount >= 20000 THEN '優良'
            ELSE '一般'
        END AS segment
    FROM user_total
)
SELECT
    segment AS "セグメント",
    COUNT(*) AS "人数",
    SUM(total_amount) AS "合計購入額"
FROM segmented
GROUP BY segment
ORDER BY CASE segment WHEN 'VIP' THEN 1 WHEN '優良' THEN 2 ELSE 3 END;

```

SQL

F-7 商品別売上 + 商品別販売数を別 CTE で結合

応用

商品ごとの売上と販売数を別々のCTEで計算し、JOINで結合してください。

▼ 解答

```
WITH item_revenue AS (  
    SELECT items.id, items.name, SUM(items.price) AS revenue  
    FROM items  
    INNER JOIN sales_records ON items.id = sales_records.item_id  
    GROUP BY items.id, items.name  
) ,  
item_count AS (  
    SELECT item_id, COUNT(*) AS sales_count  
    FROM sales_records  
    GROUP BY item_id  
)  
SELECT  
    item_revenue.name AS "商品名",  
    item_revenue.revenue AS "売上額",  
    item_count.sales_count AS "販売数"  
FROM item_revenue  
INNER JOIN item_count ON item_revenue.id = item_count.item_id  
ORDER BY item_revenue.revenue DESC;
```

F-8 月次トレンド + 前月比 を CTE で組み立てる

応用

月次売上トレンドと前月比を、CTE を使って段階的に組み立ててください。

▼ 解答

```
WITH monthly_sales AS (  
    SELECT DATE_TRUNC('month', sales_records.purchased_at) AS month, SUM(items.price)  
    AS sales  
    FROM sales_records  
    INNER JOIN items ON sales_records.item_id = items.id  
    GROUP BY DATE_TRUNC('month', sales_records.purchased_at)  
) ,  
with_prev AS (  
    SELECT month, sales, LAG(sales) OVER (ORDER BY month) AS prev_sales  
    FROM monthly_sales  
)  
SELECT  
    month AS "月",  
    sales AS "月次売上",  
    prev_sales AS "前月売上",  
    (sales - prev_sales) * 100 / NULLIF(prev_sales, 0) AS "前月比(%)"
```

```
FROM with_prev
ORDER BY month;
```

📘 トピック G : 型変換 (CAST)

G-1 数値を文字列に変換

基礎

商品IDを文字列型に変換して取得してください。

▼ 解答

```
SELECT
  CAST(id AS VARCHAR) AS "商品ID(文字列)",
  name AS "商品名"
FROM items;
```

SQL

G-2 文字列を数値に変換

基礎

'123' という文字列を整数に変換してください。

▼ 解答

```
SELECT CAST('123' AS INTEGER) AS "数値化";
```

SQL

G-3 文字列を日付に変換

基礎

'2024-01-15' という文字列を日付型に変換してください。

▼ 解答

```
SELECT CAST('2024-01-15' AS DATE) AS "日付";
```

SQL

G-4 整数除算の罣を解消

基礎

1 / 2 が 0.5 を返すように書き直してください。

▼ 解答

```
SELECT CAST(1 AS DECIMAL) / 2 AS "結果";
```

SQL

または：

```
SELECT 1.0 / 2 AS "結果";
```

SQL

G-5 商品IDを6桁0埋め文字列に

応用

商品IDを '000001' のような6桁の0埋め文字列に変換してください。

▼ 解答

```
SELECT
  LPAD(CAST(id AS VARCHAR), 6, '0') AS "商品コード",
  name AS "商品名"
FROM items;
```

SQL

G-6 文字列で格納された日付からの月次集計

応用

仮に purchased_at が文字列として格納されているとして、月単位で売上を集計してください。

▼ 解答

```
SELECT
  DATE_TRUNC('month', CAST(sales_records.purchased_at AS DATE)) AS "月",
  SUM(items.price) AS "月次売上"
FROM sales_records
INNER JOIN items ON sales_records.item_id = items.id
GROUP BY DATE_TRUNC('month', CAST(sales_records.purchased_at AS DATE))
ORDER BY "月";
```

SQL

G-7 粗利率を小数で正確に計算

応用

商品ごとの粗利率を、小数2桁まで正確に計算してください。

▼ 解答

```
SELECT
  name AS "商品名",
  price AS "販売価格",
  cost AS "原価",
  CAST(price - cost AS DECIMAL) / NULLIF(price, 0) AS "粗利率"
FROM items;
```

SQL

Phase 2 総合問題

P2-総合1 月次新規顧客数とリテンション率

総合

月ごとに、その月に初めて購入したユーザー数（新規）と過去にも購入したユーザー数（リピート）を集計。

▼ 解答

```
WITH first_purchase AS (
  SELECT user_id, MIN(purchased_at) AS first_date
  FROM sales_records
  GROUP BY user_id
),
classified AS (
  SELECT
    sales_records.user_id,
    DATE_TRUNC('month', sales_records.purchased_at) AS month,
    CASE
      WHEN DATE_TRUNC('month', sales_records.purchased_at) = DATE_TRUNC('month',
first_purchase.first_date)
      THEN '新規'
      ELSE 'リピート'
    END AS user_type
  FROM sales_records
  INNER JOIN first_purchase ON sales_records.user_id = first_purchase.user_id
)
SELECT
  month AS "月",
  COUNT(DISTINCT CASE WHEN user_type = '新規' THEN user_id END) AS "新規顧客数",
  COUNT(DISTINCT CASE WHEN user_type = 'リピート' THEN user_id END) AS "リピート顧客数"
FROM classified
```

SQL

```
GROUP BY month
ORDER BY month;
```

P2-総合2 前月比売上ランキング

総合

商品ごとの月次売上と、前月比成長率、その月の成長率ランキングを出してください。

▼ 解答

```
WITH monthly_item_sales AS (
    SELECT items.id, items.name,
           DATE_TRUNC('month', sales_records.purchased_at) AS month,
           SUM(items.price) AS sales
    FROM items
    INNER JOIN sales_records ON items.id = sales_records.item_id
    GROUP BY items.id, items.name, DATE_TRUNC('month', sales_records.purchased_at)
),
with_prev AS (
    SELECT name, month, sales,
           LAG(sales) OVER (PARTITION BY id ORDER BY month) AS prev_sales
    FROM monthly_item_sales
)
SELECT
    month AS "月",
    name AS "商品名",
    sales AS "月次売上",
    prev_sales AS "前月売上",
    (sales - prev_sales) * 100 / NULLIF(prev_sales, 0) AS "成長率(%)",
    RANK() OVER (PARTITION BY month ORDER BY (sales - prev_sales) * 100 /
    NULLIF(prev_sales, 0) DESC) AS "月内成長率順位"
FROM with_prev
WHERE prev_sales IS NOT NULL
ORDER BY month, "月内成長率順位";
```

Phase 3：中優先トピック

トピック H：文字列操作

H-1 連結 (CONCAT)

基礎

商品IDと商品名を '1: スカート' のように連結して表示してください。

▼ 解答

```
SELECT
  CONCAT(id, ': ', name) AS "商品表示名"
FROM items;
```

SQL

H-2 部分文字列の取得

基礎

商品名の最初の3文字だけを取得してください。

▼ 解答

```
SELECT
  name AS "商品名",
  SUBSTRING(name, 1, 3) AS "先頭3文字"
FROM items;
```

SQL

H-3 文字列の置換

基礎

商品名の中の「Tシャツ」を「ティーシャツ」に置き換えてください。

▼ 解答

```
SELECT
  name AS "元の商品名",
  REPLACE(name, 'Tシャツ', 'ティーシャツ') AS "置換後"
FROM items;
```

SQL

H-4 大文字化／小文字化

基礎

商品名を大文字化、小文字化、それぞれ取得してください。

▼ 解答

```
SELECT
  name AS "商品名",
  UPPER(name) AS "大文字",
  LOWER(name) AS "小文字"
FROM items;
```

SQL

H-5 0埋め (LPAD)

基礎

商品IDを5桁の0埋め文字列に変換してください。

▼ 解答

```
SELECT
  id AS "元のID",
  LPAD(CAST(id AS VARCHAR), 5, '0') AS "0埋め後"
FROM items;
```

SQL

H-6 文字列の分割 (SPLIT_PART)

基礎

仮に '山田-太郎' のような文字列を '-' で分割し、姓と名に分けてください。

▼ 解答

```
SELECT
  '山田-太郎' AS "元の文字列",
  SPLIT_PART('山田-太郎', '-', 1) AS "姓",
  SPLIT_PART('山田-太郎', '-', 2) AS "名";
```

SQL

H-7 メールアドレスからドメインを抽出

応用

仮に 'taro@example.com' から '@' 以降のドメイン部分を取り出してください。

▼ 解答

```
SELECT
  'taro@example.com' AS "メール",
  SPLIT_PART('taro@example.com', '@', 2) AS "ドメイン";
```

SQL

H-8 電話番号のハイフン除去

応用

'090-1234-5678' のような電話番号からハイフンを除去してください。

▼ 解答

```
SELECT
  '090-1234-5678' AS "元",
  REPLACE('090-1234-5678', '-', '') AS "ハイフン除去";
```

SQL

H-9 商品コードの正規化

応用

商品IDを以下の形式に変換してください: 'ITEM-000001'

▼ 解答

```
SELECT
  id AS "元のID",
  CONCAT('ITEM-', LPAD(CAST(id AS VARCHAR), 6, '0')) AS "正規化後"
FROM items;
```

SQL

H-10 氏名の姓名分割

応用

仮に users.name が '山田 太郎' のように半角スペース区切りで格納されているとして、姓と名を別カラムで取得してください。

▼ 解答

```
SELECT
  name AS "氏名",
  SPLIT_PART(name, ' ', 1) AS "姓",
  SPLIT_PART(name, ' ', 2) AS "名"
FROM users;
```

SQL

トピック I：縦結合 (UNION)

I-1 2つのテーブルを UNION ALL で結合

基礎

`items` と `users` から、IDと名前だけを取得し、それぞれの結果を縦に結合してください。

▼ 解答

```
SELECT id, name, 'items' AS source FROM items
UNION ALL
SELECT id, name, 'users' AS source FROM users;
```

SQL

I-2 UNION と UNION ALL の違いを体感

基礎

UNION と UNION ALL でそれぞれが何件返すか確認し、違いを説明してください。

▼ 解答

- UNION → 重複を排除
- UNION ALL → 重複も含めて全部

ポイント：UNION は重複排除のために裏でソート処理があり遅い。実務では基本的に UNION ALL を使う。

I-3 チャネル名を付与した結合

基礎

`items` から商品名、`users` からユーザー名を、それぞれに「種別」カラムを付けて縦に結合してください。

▼ 解答

```
SELECT name, '商品' AS "種別" FROM items
UNION ALL
SELECT name, 'ユーザー' AS "種別" FROM users;
```

SQL

I-4 複数の販売チャネル統合

応用

仮に `offline_sales` と `online_sales` の2つのテーブルがあるとして、両方を縦に結合し、チャネルラベル付きで取得してください。

▼ 解答

```
SELECT user_id, amount, '実店舗' AS "チャネル"
FROM offline_sales
UNION ALL
SELECT user_id, amount, 'EC' AS "チャネル"
FROM online_sales;
```

SQL

I-5 月次売上に「合計行」を追加

応用

月次売上の最後に、全期間の合計行を追加してください。

▼ 解答

```
WITH monthly AS (
  SELECT DATE_TRUNC('month', sales_records.purchased_at) AS month, SUM(items.price)
  AS sales
  FROM sales_records
  INNER JOIN items ON sales_records.item_id = items.id
  GROUP BY DATE_TRUNC('month', sales_records.purchased_at)
)
SELECT CAST(month AS VARCHAR) AS "月", sales AS "月次売上"
FROM monthly
UNION ALL
SELECT '合計' AS "月", SUM(sales) AS "月次売上"
FROM monthly
ORDER BY "月";
```

SQL

■ トピック J: 名寄せ実装パターン

J-1 同じメールアドレスを GROUP BY で1行に集約

基礎

`users_raw` から、メールアドレスごとにレコードを集約し、件数とともに表示してください。

▼ 解答

```
SELECT
  email,
  COUNT(*) AS "重複件数",
  MAX(name) AS "代表者名",
  MIN(updated_at) AS "最古更新日",
  MAX(updated_at) AS "最新更新日"
FROM users_raw
WHERE email IS NOT NULL
GROUP BY email
ORDER BY "重複件数" DESC;
```

SQL

J-2 名寄せキーの作成 (COALESCE)

基礎

「メール → 電話 → name」の優先順位で名寄せキーを作成してください。

▼ 解答

```
SELECT
  record_id, name, email, phone,
  COALESCE(email, phone, name) AS "名寄せキー"
FROM users_raw;
```

SQL

J-3 ROW_NUMBER で優先度1位のレコードのみ採用

基礎

メールアドレスが同じレコードが複数ある場合、最新の `updated_at` のレコードだけを残してください。

▼ 解答

```
SELECT *
FROM (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY email
      ORDER BY updated_at DESC
    )
```

SQL

```
) AS rn
FROM users_raw
WHERE email IS NOT NULL
) AS ranked
WHERE rn = 1;
```

ポイント：名寄せの中で最も実務頻出のパターン。「同じキーの中で優先度1位を残す」。

J-4 最新の更新レコードを正として残す

応用

メールアドレスごとに最新レコードを残し、CTE を使って整理してください。

▼ 解答

```
WITH ranked AS (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY email
      ORDER BY updated_at DESC
    ) AS rn
  FROM users_raw
  WHERE email IS NOT NULL
)
SELECT record_id, name, email, phone, updated_at
FROM ranked
WHERE rn = 1
ORDER BY email;
```

SQL

J-5 複数キー（メール OR 電話）で同一人物判定

応用

メール優先、なければ電話で名寄せキーを作成してください。

▼ 解答

```
WITH normalized AS (
  SELECT record_id, name, email, phone,
    COALESCE(email, phone) AS match_key
  FROM users_raw
)
SELECT
  match_key AS "名寄せキー",
  COUNT(*) AS "重複数",
  STRING_AGG(CAST(record_id AS VARCHAR), ',') AS "統合候補ID"
```

SQL

```
FROM normalized
WHERE match_key IS NOT NULL
GROUP BY match_key
HAVING COUNT(*) > 1
ORDER BY "重複数" DESC;
```

J-6 名寄せ前の正規化（全角半角統一・空白除去）

応用

name から空白を除去し、すべて小文字化したものを「正規化キー」として作成してください。

▼ 解答

```
SELECT
  record_id,
  name AS "元の氏名",
  LOWER(REPLACE(REPLACE(name, ' ', ''), ' ', '')) AS "正規化キー"
FROM users_raw;
```

SQL

J-7 名寄せ後の重複数を集計

応用

正規化キーが同じレコード数を集計し、重複が多いキーから順に表示してください。

▼ 解答

```
WITH normalized AS (
  SELECT
    LOWER(REPLACE(REPLACE(name, ' ', ''), ' ', '')) AS norm_key,
    record_id
  FROM users_raw
)
SELECT
  norm_key AS "正規化キー",
  COUNT(*) AS "重複数"
FROM normalized
GROUP BY norm_key
HAVING COUNT(*) > 1
ORDER BY "重複数" DESC;
```

SQL

J-8 名寄せ前後の件数比較レポート

応用

元レコード数・メール一意件数・正規化氏名キー一意件数を1行に並べてください。

▼ 解答

```
SELECT
COUNT(*) AS "元レコード数",
COUNT(DISTINCT email) AS "メール一意件数",
COUNT(DISTINCT LOWER(REPLACE(REPLACE(name, ' ', ''), ' ', ''))) AS "正規化氏名一意件数"
FROM users_raw;
```

SQL

J-9 顧客マスタ統合

応用

優先順位は 'CRM' > 'Form' > 'CSV' の順で同一人物の代表レコードを選んでください。

▼ 解答

```
WITH ranked AS (
SELECT *,
ROW_NUMBER() OVER (
PARTITION BY email
ORDER BY
CASE source
WHEN 'CRM' THEN 1
WHEN 'Form' THEN 2
WHEN 'CSV' THEN 3
ELSE 4
END,
updated_at DESC
) AS rn
FROM users_raw
WHERE email IS NOT NULL
)
SELECT record_id, name AS "氏名", email AS "メール", phone AS "電話",
source AS "採用ソース", updated_at AS "最終更新"
FROM ranked
WHERE rn = 1;
```

SQL

Phase 3 総合問題

P3-総合1 ECサイトの月次会員レポート

総合

新規会員数・アクティブ会員数・復活会員数を月別に集計してください。

▼ 解答

```
WITH first_purchase AS (  
    SELECT user_id, MIN(purchased_at) AS first_date  
    FROM sales_records  
    GROUP BY user_id  
) ,  
purchase_with_prev AS (  
    SELECT  
        user_id, purchased_at,  
        LAG(purchased_at) OVER (PARTITION BY user_id ORDER BY purchased_at) AS prev_date  
    FROM sales_records  
) ,  
classified AS (  
    SELECT  
        p.user_id, p.purchased_at,  
        DATE_TRUNC('month', p.purchased_at) AS month,  
        CASE  
            WHEN DATE_TRUNC('month', p.purchased_at) = DATE_TRUNC('month', f.first_date)  
            THEN '新規'  
            WHEN p.prev_date IS NULL THEN 'リピート'  
            WHEN p.purchased_at - p.prev_date > 180 THEN '復活'  
            ELSE 'リピート'  
        END AS user_type  
    FROM purchase_with_prev p  
    INNER JOIN first_purchase f ON p.user_id = f.user_id  
)  
SELECT  
    month AS "月",  
    COUNT(DISTINCT CASE WHEN user_type = '新規' THEN user_id END) AS "新規会員数",  
    COUNT(DISTINCT user_id) AS "アクティブ会員数",  
    COUNT(DISTINCT CASE WHEN user_type = '復活' THEN user_id END) AS "復活会員数"  
FROM classified  
GROUP BY month  
ORDER BY month;
```

SQL

P3-総合2 顧客LTV分析

総合

各ユーザーのLTV、購入期間、平均購入間隔、ステータスを分析してください。

▼ 解答

```
WITH user_purchase AS (  
    SELECT
```

SQL


```

users.id, users.name, users.age, users.gender,
COUNT(sales_records.id) AS purchase_count,
SUM(items.price) AS ltv,
MIN(sales_records.purchased_at) AS first_date,
MAX(sales_records.purchased_at) AS last_date,
MAX(sales_records.purchased_at) - MIN(sales_records.purchased_at) AS active_days
FROM users
INNER JOIN sales_records ON users.id = sales_records.user_id
INNER JOIN items ON sales_records.item_id = items.id
GROUP BY users.id, users.name, users.age, users.gender
)
SELECT
name AS "氏名", age AS "年齢",
CASE gender WHEN 0 THEN 'メンズ' WHEN 1 THEN 'レディース' END AS "性別",
purchase_count AS "購入回数",
ltv AS "LTV",
first_date AS "初回購入日",
last_date AS "最終購入日",
active_days AS "アクティブ日数",
active_days / NULLIF(purchase_count - 1, 0) AS "平均購入間隔(日)",
CASE
    WHEN ltv >= 50000 THEN 'VIP'
    WHEN ltv >= 20000 THEN '優良'
    WHEN ltv >= 5000 THEN '一般'
    ELSE 'ライト'
END AS "ステータス"
FROM user_purchase
ORDER BY ltv DESC;

```

P3-総合3 マルチチャネル統合の名寄せ+分析

総合

ソース別の登録件数・名寄せ後のユニーク顧客数・ソース別の重複率を集計してください。

▼ 解答

```

WITH source_stats AS (
    SELECT source, COUNT(*) AS source_count
    FROM users_raw
    GROUP BY source
),
total_stats AS (
    SELECT COUNT(*) AS total_records, COUNT(DISTINCT email) AS unique_emails
    FROM users_raw
    WHERE email IS NOT NULL
)
SELECT
    source_stats.source AS "ソース",

```

SQL

```
source_stats.source_count AS "登録件数",  
total_stats.unique_emails AS "名寄せ後ユニーク数",  
(source_stats.source_count - total_stats.unique_emails) * 100 /  
NULLIF(source_stats.source_count, 0) AS "重複率(%)"  
FROM source_stats, total_stats  
ORDER BY source_stats.source_count DESC;
```

 全体総合問題 総合問題

総-1 月次売上ダッシュボード

総合

月次売上額・前月売上・前月比成長率・累計売上・月内1位商品を一つのクエリで集計。

▼ 解答

```
WITH monthly AS (SQL
    SELECT DATE_TRUNC('month', sales_records.purchased_at) AS month, SUM(items.price)
AS sales
    FROM sales_records
    INNER JOIN items ON sales_records.item_id = items.id
    GROUP BY DATE_TRUNC('month', sales_records.purchased_at)
),
monthly_top_item AS (
    SELECT
        DATE_TRUNC('month', sales_records.purchased_at) AS month,
        items.name,
        SUM(items.price) AS item_sales,
        ROW_NUMBER() OVER (
            PARTITION BY DATE_TRUNC('month', sales_records.purchased_at)
            ORDER BY SUM(items.price) DESC
        ) AS rn
    FROM sales_records
    INNER JOIN items ON sales_records.item_id = items.id
    GROUP BY DATE_TRUNC('month', sales_records.purchased_at), items.id, items.name
)
SELECT
    monthly.month AS "月",
    monthly.sales AS "月次売上",
    LAG(monthly.sales) OVER (ORDER BY monthly.month) AS "前月売上",
    (monthly.sales - LAG(monthly.sales) OVER (ORDER BY monthly.month)) * 100 /
NULLIF(LAG(monthly.sales) OVER (ORDER BY monthly.month), 0) AS "前月比(%)",
    SUM(monthly.sales) OVER (ORDER BY monthly.month) AS "累計売上",
    monthly_top_item.name AS "月内1位商品"
FROM monthly
LEFT JOIN monthly_top_item ON monthly.month = monthly_top_item.month AND
```

```
monthly_top_item.rn = 1
ORDER BY monthly.month;
```

総-2 顧客のRFM分析

総合

R/F/M を NTILE で5段階に分け、合計スコアを算出。

▼ 解答

```
WITH max_date AS (SELECT MAX(purchased_at) AS latest FROM sales_records),
rfm AS (
  SELECT
    users.id, users.name,
    max_date.latest - MAX(sales_records.purchased_at) AS recency,
    COUNT(sales_records.id) AS frequency,
    SUM(items.price) AS monetary
  FROM users
  INNER JOIN sales_records ON users.id = sales_records.user_id
  INNER JOIN items ON sales_records.item_id = items.id
  CROSS JOIN max_date
  GROUP BY users.id, users.name, max_date.latest
),
scored AS (
  SELECT
    name AS "氏名",
    recency AS "R(日数)",
    frequency AS "F(回数)",
    monetary AS "M(金額)",
    NTILE(5) OVER (ORDER BY recency) AS r_score_inv,
    NTILE(5) OVER (ORDER BY frequency DESC) AS f_score,
    NTILE(5) OVER (ORDER BY monetary DESC) AS m_score
  FROM rfm
)
SELECT
  "氏名", "R(日数)", "F(回数)", "M(金額)",
  6 - r_score_inv AS "R評価",
  6 - f_score AS "F評価",
  6 - m_score AS "M評価",
  (6 - r_score_inv) + (6 - f_score) + (6 - m_score) AS "RFM合計"
FROM scored
ORDER BY "RFM合計" DESC;
```

総-3 商品のABC分析

総合

累計70%以下=A、90%以下=B、それ以上=C で分類。

▼ 解答

```
WITH item_sales AS (SQL
  SELECT items.id, items.name, SUM(items.price) AS sales
  FROM items
  INNER JOIN sales_records ON items.id = sales_records.item_id
  GROUP BY items.id, items.name
),
ranked AS (
  SELECT
    name, sales,
    SUM(sales) OVER (ORDER BY sales DESC) AS cumulative_sales,
    SUM(sales) OVER () AS total_sales
  FROM item_sales
)
SELECT
  name AS "商品名",
  sales AS "売上額",
  cumulative_sales AS "累計売上",
  cumulative_sales * 100 / total_sales AS "累計比率(%)",
  CASE
    WHEN cumulative_sales * 100 / total_sales <= 70 THEN 'A'
    WHEN cumulative_sales * 100 / total_sales <= 90 THEN 'B'
    ELSE 'C'
  END AS "ABC区分"
FROM ranked
ORDER BY sales DESC;
```

総-4 コホート分析

総合

初回購入月でコホートに分け、各コホートのNヶ月後継続率を計算。

▼ 解答

```
WITH first_purchase AS (SQL
  SELECT user_id, DATE_TRUNC('month', MIN(purchased_at)) AS cohort_month
  FROM sales_records
  GROUP BY user_id
),
cohort_activity AS (
  SELECT
    fp.cohort_month,
    sr.user_id,
    DATE_TRUNC('month', sr.purchased_at) AS active_month,
```

```

    EXTRACT(MONTH FROM AGE(sr.purchased_at, fp.cohort_month)) AS months_since
FROM sales_records sr
INNER JOIN first_purchase fp ON sr.user_id = fp.user_id
),
cohort_size AS (
    SELECT cohort_month, COUNT(DISTINCT user_id) AS cohort_users
    FROM first_purchase
    GROUP BY cohort_month
)
SELECT
    ca.cohort_month AS "コホート月",
    ca.months_since AS "経過月",
    COUNT(DISTINCT ca.user_id) AS "アクティブユーザー数",
    cs.cohort_users AS "コホート総数",
    COUNT(DISTINCT ca.user_id) * 100 / cs.cohort_users AS "継続率(%)"
FROM cohort_activity ca
INNER JOIN cohort_size cs ON ca.cohort_month = cs.cohort_month
GROUP BY ca.cohort_month, ca.months_since, cs.cohort_users
ORDER BY ca.cohort_month, ca.months_since;

```

総-5 異常値検知 (移動平均比較)

総合

日次売上が7日移動平均の150%以上または50%以下の日を異常値として検出。

▼ 解答

```

WITH daily AS (
    SELECT sales_records.purchased_at AS day, SUM(items.price) AS daily_sales
    FROM sales_records
    INNER JOIN items ON sales_records.item_id = items.id
    GROUP BY sales_records.purchased_at
),
with_avg AS (
    SELECT day, daily_sales,
        AVG(daily_sales) OVER (
            ORDER BY day
            ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
        ) AS moving_avg_7d
    FROM daily
)
SELECT
    day AS "日付",
    daily_sales AS "日次売上",
    moving_avg_7d AS "7日移動平均",
    daily_sales * 100 / NULLIF(moving_avg_7d, 0) AS "比率(%)",
    CASE
        WHEN daily_sales >= moving_avg_7d * 1.5 THEN '⚠ 急増'
    END

```

SQL

```

    WHEN daily_sales <= moving_avg_7d * 0.5 THEN '⚠ 急減'
    ELSE '正常'
  END AS "判定"
FROM with_avg
WHERE daily_sales >= moving_avg_7d * 1.5
    OR daily_sales <= moving_avg_7d * 0.5
ORDER BY day;

```

総-6 名寄せされた顧客マスタを使った属性別分析

総合

users_raw を名寄せして、ソース別ユニーク顧客数・全体ユニーク顧客数・名寄せ前後の件数比較を集計。

▼ 解答

```

WITH ranked AS (
    SELECT *,
        ROW_NUMBER() OVER (PARTITION BY email ORDER BY updated_at DESC) AS rn
    FROM users_raw
    WHERE email IS NOT NULL
),
unified AS (SELECT * FROM ranked WHERE rn = 1),
source_summary AS (
    SELECT source, COUNT(*) AS unique_count
    FROM unified
    GROUP BY source
)
SELECT
    source_summary.source AS "ソース",
    source_summary.unique_count AS "名寄せ後ユニーク数",
    (SELECT COUNT(*) FROM users_raw WHERE source = source_summary.source) AS "元レコード数",
    (SELECT COUNT(*) FROM users_raw WHERE source = source_summary.source) -
    source_summary.unique_count AS "重複削減数"
FROM source_summary
UNION ALL
SELECT
    '全体合計' AS "ソース",
    (SELECT COUNT(*) FROM unified) AS "名寄せ後ユニーク数",
    (SELECT COUNT(*) FROM users_raw WHERE email IS NOT NULL) AS "元レコード数",
    (SELECT COUNT(*) FROM users_raw WHERE email IS NOT NULL) - (SELECT COUNT(*) FROM
unified) AS "重複削減数"
ORDER BY "ソース";

```

解答を見る前に、まず自力で書いてみる。
書けたら答え合わせ、違えば原因を書き出す。
これだけで力がつきます。